

Preregistered Paired Evaluation of a Deterministic Verifier Pipeline for LLM-Generated Network Configuration Changes — Non-informative Result under Shared-Generation Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered paired experiment (cand-2026-01-prereg-v1) that tested whether a deterministic verifier pipeline (generate → deterministic_netops_validator_v5 → optional repair) reduces the rate of verifier-detected unsafe intent→configuration proposals produced by a hosted LLM on synthetic NetOps tasks. Design: N=200 preregistered unique intent→configuration tasks in a paired design comparing (a) baseline LLM-only and (b) guarded pipeline (gpt-5-mini + deterministic_netops_validator_v5). Execution used shared_initial_candidate generation semantics (one LLM call per task) producing model_calls_used = 200 (preregistered independent per-arm generation would have used up to 400). Observed (adversarial cluster): baseline SAFE = 200/200; guarded SAFE = 200/200 (paired contingency n11 = 200, n10 = 0, n01 = 0). Estimated paired SAFE-rate difference = 0.0; exact McNemar two-sided $p = 1.0$; paired bootstrap 95% CI = [0.0, 0.0] (resamples = 10000, seed = 1729). Because identical per-pair generations were produced and the observed baseline unsafe rate was 0%, the preregistered causal hypothesis (that the deterministic verifier reduces unsafe proposals) was not testable in this execution. We publish the complete preregistered run and artifacts, provide an artifact-grounded diagnosis of testability failure, and give concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial injection calibration, and exploratory ROC reserves) for informative repeats. All execution and analysis artifacts cited here are archived in the supplied bundle.

I. INTRODUCTION

Automating human-intent → device-configuration translation with large language models (LLMs) is an active NetOps research thrust because it can reduce human effort and speed maintenance tasks. Yet incorrectly specified or misordered changes can cause outages, policy violations, or security exposures. A commonly proposed mitigation is tool-augmentation: couple an LLM generator to deterministic validators, sandboxed simulators, or constrained executors in a generate-validate-repair pipeline so that proposed changes are checked before execution.

We preregistered and executed a confirmatory paired experiment (cand-2026-01-prereg-v1) to quantify whether deterministic verifier augmentation causally reduces the frequency of verifier-detected unsafe proposals from a hosted LLM under both benign and adversarial prompt clusters. The preregistered estimand was the paired SAFE-rate difference (guarded minus

baseline), tested by an exact McNemar two-sided test with a paired bootstrap CI (10000 resamples, seed = 1729). The design sampled N=200 tasks using a deterministic generator and a paired comparison across two pipelines. Two generation semantics were permitted in the preregistration: independent per-arm generation (one LLM call per arm per task) or shared_initial_candidate (one shared LLM call per task scored by both arms). The executed run used shared_initial_candidate to conserve model-call budget.

Key outcome: under the executed shared semantics the sampled LLM outputs were scored SAFE by the deterministic validator in every confirmatory episode (baseline SAFE = 200/200; guarded SAFE = 200/200). This concordant ceiling outcome (n11 = 200; n10 = n01 = n00 = 0) yields an estimate of zero paired difference and an exact McNemar $p = 1.0$. Because the observed baseline unsafe rate was 0% and the same candidate was used in both arms, the preregistered causal hypothesis could not be meaningfully evaluated. The manuscript therefore focuses on three contributions grounded in archived artifacts: (1) a complete, deterministic preregistered run published as reproducible artifacts; (2) a precise, artifact-grounded diagnosis of why this execution was non-informative for the confirmatory estimand; and (3) concrete, artifact-supported design recommendations that would enable informative repeats under the same preregistration framework.

II. RELATED WORK

Recent surveys and prototypes explore LLMs for NetOps tasks including intent translation, configuration generation, and remediation planning; these works emphasize both potential productivity gains and safety concerns when LLMs propose executable changes [openalex:W7161354925; openalex:W7170714602; TechRxiv:177004277]. Prior empirical and architectural work shows that augmenting LLMs with deterministic tools (verifiers, simulators, guarded tool schemas) often improves measured correctness on curated task sets and can reduce constraint violations in produce-validate pipelines [openalex:W4412888330; openalex:W7161354925]. Red-teaming, guardrail, and MCP/schema proposals highlight trade-offs between missed unsafe proposals and increased false refusals, and they call for workflow-centred, replayable

evaluations with deterministic sandboxing and archived traces [TechRxiv:177006109; TechRxiv:177004277].

Our study is not a new verification algorithm or a multi-model leaderboard. Instead, it complements existing work by empirically interrogating how concrete experimental semantics—particularly generation budgeting and shared vs independent generation—interact with a paired estimand and can render a confirmatory design untestable. We therefore situate our contribution as a reproducible, methodological diagnosis that bridges architecture proposals and practical evaluation design.

III. METHODOLOGY

Preregistration and estimand. The experiment was preregistered as cand-2026-01-prereg-v1. Primary estimand: `paired_success_rate_difference_guarded_minus_baseline` (`guarded` - `baseline`) on binary SAFE indicators obtained from a deterministic sandbox validator. Confirmatory test: exact two-sided McNemar implemented by `paired_binary_analysis_v1`; confidence intervals: paired bootstrap percentile (10000 resamples, seed = 1729). The preregistration specified a paired binary design with $N=200$ tasks, planned paired episodes = 400, and a power plan that targeted detecting an approximate 7 percentage-point reduction in unsafe proposals with 80% power assuming a baseline unsafe rate $\approx 10\text{--}15\%$.

Task generator, scope, and seeds. A deterministic generator (`deterministic_netops_task_generator_v5`) produced 200 synthetic intent→configuration tasks constrained to single-AS multi-device topologies and limited configuration fields (`admin`, `mtu`, `vlan`). Generator seeds were deterministic (per-task `generation_seed` 1..200; the RNG seed for the generator family was preregistered) and each task manifest included `initial_state`, `expected_final_state`, explicit per-task workflow policies (`required_sequence`), and difficulty metadata. Tasks were clustered by benign vs adversarial prompt transforms using a preregistered obfuscation template set to increase adversarial difficulty; the adversarial cluster was included in confirmatory items per the preregistration.

Pipelines compared. Baseline: accept the LLM candidate (no runtime deterministic enforcement) and score post hoc with `deterministic_netops_validator_v5`. Guarded: validate the same LLM candidate in a deterministic sandbox using `deterministic_netops_validator_v5`; the validator emitted structured traces (`validation_before/after`, violation lists) and optionally could invoke one automatic repair call per the preregistration. For confirmatory episodes no repairs were applied (repair fields are null in scoring JSONs) so the guarded result reflects pure validation rather than repair-induced change.

Generation semantics and model budgeting. The preregistered plan allowed two generation semantics: independent per-arm generation (two model calls per task) or `shared_initial_candidate` (one model call per task whose candidate is scored by both arms). The executed run used `shared_initial_candidate` to conserve hosted model budget. Planned maximum_model_calls for independent generation =

400; executed `model_calls_used` = 200. This single shared candidate per task meant that, absent repair, the two arms would always score the same candidate and could not produce discordant paired outcomes arising from arm-level stochasticity.

Scoring and analysis pipeline. `Deterministic_netops_validator_v5` applied `full_intent_constraint_validation` to each candidate and emitted a deterministic valid flag and detailed violation lists. Confirmatory binary outcomes were derived from `validator.valid` (1 = SAFE, 0 = UNSAFE). Analysis used `paired_binary_analysis_v1` to compute the 2×2 contingency counts ($n_{11}, n_{10}, n_{01}, n_{00}$), exact McNemar two-sided p-value, and paired bootstrap CI (10000 resamples, seed = 1729). Missingness and retry rules from the preregistration were applied: one automatic retry per failed model call; contaminated or nondeterministic tasks are excluded and replaced. Deterministic reconciliation checks and provider audit were recorded as analysis artifacts and used to certify that no technical failures, exclusions, or nondeterminism occurred in the confirmatory run.

IV. RESULTS

Execution accounting and provider audit. The execution manifest reports `completed_episode_count` = 400 (200 paired episodes), `failed_episode_count` = 0, and `model_calls_used` = 200, consistent with the executed `shared_initial_candidate` semantics. Provider-level auditing recorded `hosted_model_call_event_count` = 200 and `cache_miss_count` = 200; cross-condition cache reuse was not observed. No technical failures, exclusions, or nondeterministic verifier behaviors were recorded in the confirmatory set, and deterministic reconciliation checks passed, confirming that the archived episode and scoring artifacts correspond to a single, reproducible run.

Primary confirmatory outcomes and contingency. The deterministic validator produced SAFE labels for every confirmatory episode: baseline SAFE successes = 200/200 and guarded SAFE successes = 200/200. The resulting paired contingency counts are $n_{11} = 200$ (SAFE in both arms), $n_{10} = 0$ (SAFE guarded only), $n_{01} = 0$ (SAFE baseline only), and $n_{00} = 0$ (UNSAFE in both). From these counts the paired SAFE-rate difference (`guarded` - `baseline`) equals 0.0. The preregistered exact McNemar two-sided test computed $p = 1.0$; the paired bootstrap percentile 95% confidence interval is degenerate at $[0.0, 0.0]$ (resamples = 10000; `bootstrap_seed` = 1729). These numerical values are direct outputs of `paired_binary_analysis_v1` and are archived in the analysis artifacts (`condition_summary` and `paired_contingency_table` files).

Representative diagnostic traces. Representative scoring JSONs and validator traces illustrate why the contingency is degenerate. For multiple sampled tasks the `shared_initial_candidate` content (the raw generated configuration) is identical across baseline and guarded episodes; validator traces show `validation_before` and `validation_after`

with `violation_count = 0` and `valid = true` in every case. The `repair_provider_trace` entries are null in all confirmatory scoring JSONs, confirming that no automatic repair was invoked and that the guarded arm performed only validation. Shared response files for representative tasks contain the canonical generated configurations used by both arms, reinforcing that no arm-specific generation stochasticity existed under the executed semantics.

Missingness, contamination, and exploratory reserves. Analysis/missingness_summary documents `complete_pairs = 200` with zero failed or unscorable episodes; analysis/contamination_summary shows no flagged episodes. The preregistration reserved up to 50 exploratory tasks to estimate verifier ROC and refusal trade-offs; that exploratory reserve was not consumed prior to confirmatory execution, so no empirical ROC or refusal-rate estimates appear in the confirmatory analysis artifacts.

Statistical interpretation and inferential consequence. The observed ceiling outcome ($n_{11} = 200$ with no discordant pairs) renders the preregistered inferential machinery non-informative for the intended causal estimand: McNemar’s test relies on discordant pairs (n_{10} and n_{01}) to evaluate paired differences, and their absence produces a p-value of 1.0 and a degenerate CI at zero. This degeneracy does not demonstrate verifier effectiveness; rather, it indicates that under the executed shared generation semantics and for the sampled model outputs, there was no arm-level variation to attribute to the presence or absence of the deterministic validator. The preregistered power calculation had assumed a nonzero baseline unsafe rate ($\approx 10\text{--}15\%$) and discordant proportions; those assumptions were not met in the executed run because budgeted generation semantics constrained per-arm variability.

Archival fidelity and reproducibility pointers. All numerical summaries and the contingency table reported above are identical to the archived analysis artifacts (analysis/condition_summary.csv and analysis/paired_contingency_table.csv). Representative per-task scoring JSONs and shared response files show the identical generated candidate for baseline and guarded episodes and document validator decisions and violation lists. The execution and analysis logs capture the exact bootstrap parameters (`resamples = 10000`, `seed = 1729`) and the exact statistical outputs. Because the confirmatory run contains no technical failures or exclusions, the archived artifacts permit deterministic replay of the identical non-informative outcome; investigators seeking an informative repeat should consult these artifacts before changing preregistered design choices.

V. DISCUSSION

We expand the diagnostic interpretation and practical implications of the observed non-informative (ceiling) outcome, grounding each statement in archived execution and analysis artifacts.

Root cause synthesis. The degenerate contingency ($n_{11}=200$, $n_{10}=n_{01}=n_{00}=0$) results from the conjunction of three archived facts: (1) generation semantics: the run executed

`shared_initial_candidate` (independent_condition_generation = false) and therefore produced exactly one LLM candidate per task that both arms scored; (2) validator outcomes: deterministic_netops_validator_v5 labeled every sampled candidate as valid (baseline SAFE = 200/200); and (3) absence of repairs: `repair_provider_trace` fields are null in confirmatory scoring JSONs, so the guarded arm did not alter any candidate. These archived facts (execution/model_configuration.json, execution/execution_manifest.json, execution/scoring/*.json, analysis/paired_contingency_table.csv) together make the paired causal contrast mechanically untestable because the arms had no opportunity to disagree.

Implications for confirmatory design. A paired estimand that attributes differences to the presence of a deterministic verifier requires per-arm variation that could produce discordant outcomes. Under shared generation semantics, discordance can only arise from repair-driven changes in the guarded arm or nondeterministic validator behavior; neither occurred here. Accordingly, the preregistered power calculation—which assumed a baseline unsafe rate $\approx 10\text{--}15\%$ and discordant proportions that yield detectability with $N \approx 174$ —no longer applied. The archived model_call accounting quantifies the specific resource trade-off: independent per-arm generation would have required ≈ 400 model calls (preregistered maximum_model_calls), whereas the executed shared semantics used 200 calls (execution/execution_manifest.json). When budgets force a choice, the estimand should drive budgeting: per-arm independence is necessary to test the causal effect of adding a validator to generation.

Practical recommendations for informative repeats (artifact-grounded). All recommendations follow directly from the preregistration and execution artifacts: - Use independent per-arm generation for confirmatory paired contrasts unless the guarded arm is capable of deterministic repairs that will reliably produce candidate changes. For this study size ($N=200$), independent generation requires roughly 200 additional model calls (400 total) compared with the executed shared run (200 model_calls_used). This arithmetic is visible in the preregistered execution_contract and the executed manifest. - Prior to running the confirmatory set, consume a preregistered exploratory reserve (we reserved up to 50 tasks) to calibrate adversarial transforms until the baseline unsafe rate matches the power assumptions. Archive these tuning runs and fix the transform parameters before the confirmatory execution so the confirmatory assumptions remain preregistered and reproducible. - If repairs are part of the guarded pipeline, instrument, archive, and analyze repair_provider_trace artifacts: quantify the fraction of UNSAFE→SAFE conversions and check whether repairs introduce new violations. In our run repair traces were null; therefore repairs did not serve as a mechanism to induce discordance. - Implement a deterministic pilot check step that inspects the contingency counts on a small holdout before committing to the full confirmatory test. If the pilot produces a ceiling/floor contingency (no discordant pairs), abort and adapt along preregistered contingency-preserving paths (e.g.,

enable independent per-arm draws or strengthen adversarial transforms) rather than proceeding to a confirmatory test that will be uninformative.

Alternative estimands and complementary analyses. When budgets or policy preclude independent per-arm draws, consider changing the estimand to one that remains testable under shared semantics: (a) estimate the absolute unsafe-proposal rate of the LLM baseline (single-arm proportion) with appropriate CIs; (b) treat the guarded pipeline as an accept/reject gate and estimate refusal and post-repair SAFE rates (requires repairs and repair traces); or (c) treat the validator as a diagnostic tool and run exploratory ROC estimation (requires ground truth UNSAFE injections and the exploratory reserve). These alternative estimands shift the required resources but preserve inferential validity; the preregistration explicitly listed several secondary_estimands and exploratory items that map to these alternatives.

Threats to generalization and validator completeness. The deterministic validator used here encodes a formalized full_intent_constraint_validation for the synthetic task family and ran deterministically in the sandboxed environment; however, its completeness relative to production failure modes is limited. The synthetic tasks restrict fields to admin/mtu/vlan and single-AS topologies; consequently, zero unsafe detections here do not imply general safety in production multi-vendor or multi-AS contexts. Future work should validate validators against broader operational rule sets and consider human adjudication for ambiguous cases; such extensions must be preregistered to avoid post-hoc instrument selection.

Operational implications. From an operator perspective, strict guardrails can reduce risk but may also increase false refusals and operator workload. Our archived run did not produce refusals, so we cannot quantify that trade-off here; nevertheless, the preregistered exploratory ROC reserve was intended to measure precisely this tension and should be consumed in future repetitions. For deployment decisions, teams should balance the cost of additional model calls (for independent draws or exploratory calibration) against the operational risk of executing unverified changes; our artifacts make the per-task resource implications explicit and replayable.

Reproducibility and reuse. All artifacts supporting the above diagnosis and recommendations are archived (execution/* and analysis/*). They permit deterministic replay of the executed semantics (shared_initial_candidate), verification of model_call accounting (model_calls_used = 200 vs planned 400), and inspection of representative scoring traces that expose why no arm-level discordance occurred. We encourage future repeats to adopt the pilot-check and exploratory calibration steps described above and to record the exact decision points in the preregistration so that execution semantics remain auditable and the confirmatory estimand remains testable.

VI. LIMITATIONS

- Synthetic tasks and scope: tasks were deterministic synthetic topologies produced by deterministic_netops_task_generator_v5 and limited to single-AS,

multi-device setups; results may not generalize to production, multi-AS, or vendor-specific features.

- Generation semantics: the executed run used shared_initial_candidate (independent_condition_generation = false), producing identical per-pair generations and creating a ceiling effect when shared candidates were valid.
- Adversarial strength: the preregistered adversarial templates used in this run did not elicit unsafe outputs from gpt-5-mini on the sampled tasks; stronger adversarial cases or explicit injected unsafe examples are required to measure verifier effect.
- Single-model scope: only the declared model (gpt-5-mini) was evaluated; no multi-model generality is claimed.
- Verifier completeness: deterministic_netops_validator_v5 ran deterministically in synthetic sandboxed states; external validation of validator completeness against all real-world NetOps failure modes is not provided.
- Operational external validity: no production deployment, operator-in-the-loop workload measurement, nor multi-vendor field trials were performed; operator-cost trade-offs remain unquantified at scale.

VII. CONCLUSION

We executed a fully archived preregistered paired experiment comparing gpt-5-mini baseline generation and a deterministic verifier-augmented pipeline on 200 synthetic NetOps tasks. Under the executed shared_initial_candidate semantics and for the declared model, both arms produced zero verifier-detected unsafe proposals (paired difference = 0.0; exact McNemar $p = 1.0$; 95% CI = [0.0, 0.0]). Because identical per-pair generations were used and because the observed baseline unsafe rate was 0%, the preregistered causal hypothesis was not testable in this run. The scientific contribution is therefore methodological and reproducibility-centred: (1) a complete deterministic preregistered run with archived artifacts that permit exact replay; (2) an artifact-grounded diagnosis of why this execution was non-informative for verifier efficacy; and (3) concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial calibration, and exploratory ROC estimation) for informative repeats. The archived execution and analysis artifacts cited throughout (execution/* and analysis/*) permit deterministic replication of the diagnosis and the run outputs and should be consulted prior to attempting repeats or production extrapolation.

DISCLOSURE STATEMENT

Disclosure Statement (CNSM Agentic AI Researcher track): Declared LLM: gpt-5-mini (execution recorded in execution/model_configuration.json). Orchestration/framework: hosted_netops_gvr_v1 adapter and deterministic experiment harness (execution/execution_manifest.json). Exact initial master prompt: archived at provenance/master_prompt.txt with immutable SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15. Topic constraints and autonomy boundary: the human

Research Director selected the broad topic family (Generative AI and LLMs for NetOps) prior to the autonomy lock; after the lock the autonomous researcher performed literature review, preregistration (cand-2026-01-prereg-v1), experiment design, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revisions. Preregistration and execution: the preregistration was frozen before execution; key preregistered elements (estimand, paired design N=200, task generator, allowed generation semantics, scoring rules, and stopping rules) are recorded in cand-2026-01-prereg-v1 and archived in the manuscript bundle. Generation semantics and model-call accounting (executed): `independent_condition_generation = false` (`shared_initial_candidate` semantics) producing a single initial LLM candidate per task shared across arms; `model_calls_used = 200` while the preregistered `maximum_model_calls` allocated for independent per-arm generation = 400 (execution/execution_manifest.json and execution/model_configuration.json). Validator and scoring: `deterministic_netops_validator_v5` performed deterministic sandboxed validation and encoded outcomes as binary labels (1 = SAFE, 0 = UNSAFE) under `scoring_rule = full_intent_constraint_validation`; archived scoring JSONs (execution/scoring/*.json) contain validator traces showing `validation_before/after` and violation lists. Analysis executor: `paired_binary_analysis_v1` computed the paired contingency, exact McNemar two-sided test, and paired bootstrap CI (resamples = 10000, seed = 1729); analysis artifacts include `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv`. Deviations and restarts: no post-preregistration design changes were executed; the observed model call count reflects the executed `shared_initial_candidate` semantics. Reproducibility pointers (concise): `execution/raw_results.jsonl`, `execution/task_manifest.jsonl`, `execution/model_configuration.json`, `execution/execution_manifest.json`, `analysis/paired_contingency_table.csv`, `analysis/condition_summary.csv`, and `analysis/paired_difference_figure.svg` are archived in the supplied bundle and permit deterministic replays. Human editing: no manual human edits to technical content, analyses, or manuscript text occurred after the autonomy lock. Pre-lock development: infrastructure rehearsals occurred prior to the lock but were excluded from final empirical evidence unless reproduced under the post-lock execution.

REFERENCES

- [1] <https://arxiv.org/abs/2605.12729>
- [2] <https://doi.org/10.2139/ssrn.7132138>
- [3] <https://doi.org/10.36227/techrxiv.177004277.71795055/v1>
- [4] <https://doi.org/10.36227/techrxiv.177006109.97957916/v1>
- [5] <https://doi.org/10.1109/ojcoms.2026.3680457>
- [6] Xianren Zhang, Xianfeng Tang, Hui Liu, Zongyu Wu, Qi He, Dongwon Lee, Suhang Wang, "Divide-Verify-Refine: Can LLMs Self-align with Complex Instructions?", 2025, 10.18653/v1/2025.findings-acl.709
- [7] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Shahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026